

LAPORAN PRAKTIKUM

Inheritance, Abstract Class and Interface

Diajukan untuk memenuhi salah satu tugas praktikum Mata kuliah Pemrograman Berorientasi Objek



Disusun Oleh:
Hasya Nadana Hakim (251511040)
Jurusan Teknik Komputer dan Informatika

Program Studi D-3 Teknik Informatika
Politeknik Negeri Bandung 2026

Link Repository: <https://github.com/shiningsstars/Praktikum-PBO-W4.git>

1. Deskripsi program

Pada praktikum ini saya mencoba beberapa program Java untuk memahami konsep inheritance, method overriding, abstract class, dan sorting.

Program pertama membahas hubungan inheritance antara class Circle dan Cylinder. Class Cylinder merupakan turunan dari class Circle sehingga dapat menggunakan method dan attribute yang diwariskan. Pada program ini juga diterapkan constructor super(), overriding method getArea(), dan overriding toString().

Program kedua membuat superclass Shape dengan beberapa subclass yaitu Circle, Rectangle, dan Square. Program ini digunakan untuk memahami inheritance bertingkat serta penggunaan overriding pada method tertentu. Square merupakan subclass dari Rectangle dan menggunakan width dan length yang diwarisi dari superclass.

Program ketiga menggunakan abstract class Sortable. Class Employee dibuat sebagai subclass dari Sortable dan mengimplementasikan method compare() untuk membandingkan salary. Data employee kemudian dapat diurutkan menggunakan metode shell_sort.

2. Exercise 1 - Circle dan Cylinder

Percobaan ini bertujuan untuk memahami inheritance antara Circle sebagai superclass dan Cylinder sebagai subclass.

Source Code

Circle.java

```
public class Circle {

    private double radius;
    private String color;

    public Circle() {
        radius = 1.0;
        color = "red";
    }

    public Circle(double radius) {
        this.radius = radius;
        color = "red";
    }

    public Circle(double radius, String color) {
        this.radius = radius;
        this.color = color;
    }

    public double getRadius() {
        return radius;
    }
}
```

```

    public double getArea() {
        return radius * radius * Math.PI;
    }

    public String getColor() {
        return color;
    }

    public void setColor(String color) {
        this.color = color;
    }

    @Override
    public String toString() {
        return "Circle[radius=" + radius
            + " color=" + color + "];"
    }
}

```

Penjelasan Source Code

Cylinder menggunakan keyword `extends` untuk mewarisi class `Circle`, seperti pada `public class Cylinder extends Circle`. Constructor pada `Cylinder` memakai `super()` untuk memanggil constructor dari `Circle`. Pada method `getArea()` dilakukan overriding karena rumus luas permukaan cylinder berbeda dengan luas circle, sedangkan `super.getArea()` digunakan untuk mengambil luas circle dari superclass sebagai luas alas cylinder. Selanjutnya, method `getVolume()` menghitung volume cylinder dengan rumus $\text{volume} = \text{luas alas} \times \text{tinggi}$.

Hasil Implementasi:

```

PS C:\Users\hsyyn\Downloads\Kuliah\Smt 3\PBO\W4 PBO\Exercise 3> cd "c:\Users\hsyyn\Downloa
ds\Kuliah\Smt 3\PBO\W4 PBO\Exercise 1\" ; if ($?) { javac TestCylinder.java } ; if ($?) {
java TestCylinder }
Cylinder: radius=1.0 height=1.0 base area=12.566370614359172 volume=3.141592653589793
Cylinder: subclass of Circle[radius=1.0 color=red] height=1.0
Cylinder: radius=1.0 height=10.0 base area=69.11503837897544 volume=31.41592653589793
Cylinder: subclass of Circle[radius=1.0 color=red] height=10.0
Cylinder: radius=2.0 height=10.0 base area=150.79644737231007 volume=125.66370614359172
Cylinder: subclass of Circle[radius=2.0 color=red] height=10.0

```

3. Exercise 2 - Shape, Circle, Rectangle, dan Square

Percobaan ini digunakan untuk memahami inheritance bertingkat pada beberapa class.

3.1 Shape

Shape digunakan sebagai superclass yang memiliki atribut umum:

```

private String color;

private boolean filled;

```

Class ini memiliki constructor, getter, setter, dan method `toString()`.

3.2 Circle

Circle merupakan turunan dari Shape.

```

public class Circle extends Shape

```

Circle memiliki atribut radius. Rumus yang digunakan:

$$\text{Luas} = \pi \times r^2$$
$$\text{Keliling} = 2 \times \pi \times r$$

Method toString() juga dioverride agar informasi dari Circle dapat ditampilkan.

3.3 Rectangle

Rectangle juga merupakan turunan dari Shape.

Atribut yang digunakan:

```
private double width;
```

```
private double length;
```

Rumus luas:

$$\text{Luas} = \text{width} \times \text{length}$$

Rumus keliling:

$$\text{Keliling} = 2 \times (\text{width} + \text{length})$$

3.4 Square

Square merupakan turunan dari Rectangle.

```
public class Square extends Rectangle
```

Karena square memiliki panjang dan lebar yang sama, method setWidth() dan setLength() dioverride.

Contohnya:

```
@Override  
  
public void setWidth(double width) {  
    super.setWidth(width);  
    super.setLength(width);  
}
```

Dengan cara tersebut, perubahan nilai width akan membuat length ikut berubah.

Hasil Implementasi:

```
PS C:\Users\hsyyn\Downloads\Kuliah\Smt 3\PBO\W4 PBO\Exersice 2> cd "c:\Users\hsyyn\Downloa  
ds\Kuliah\Smt 3\PBO\W4 PBO\Exersice 2\" ; if ($?) { javac TestShape.java } ; if ($?) { jav  
a TestShape }  
Shape: color=green filled=true  
A Shape with color of green and filled  
Shape: color=yellow filled=false  
A Shape with color of yellow and Not filled  
Circle: radius=1.0 area=3.141592653589793 perimeter=6.283185307179586  
A Circle with radius=1.0, which is a subclass of A Shape with color of green and filled  
Circle: radius=2.0 area=12.566370614359172 perimeter=12.566370614359172  
A Circle with radius=2.0, which is a subclass of A Shape with color of green and filled  
Circle: radius=3.0 area=28.274333882308138 perimeter=18.84955592153876  
A Circle with radius=3.0, which is a subclass of A Shape with color of red and filled
```

4. Exercise 3 - Employee dan Manager

Percobaan ini bertujuan untuk memahami inheritance antara Employee dan Manager, terutama pada penggunaan overriding.

4.1 Employee

Class Employee memiliki beberapa atribut:

```
private String name;  
private double salary;  
private int hireday;  
private int hiremonth;  
private int hireyear;
```

Method raiseSalary() digunakan untuk menaikkan salary berdasarkan persentase.

```
public void raiseSalary(double byPercent) {  
    salary *= 1 + byPercent / 100;  
}
```

4.2 Manager

Manager merupakan subclass dari Employee.

```
public class Manager extends Employee
```

Constructor Manager menggunakan:

```
super(n, s, d, m, y);
```

untuk memanggil constructor dari Employee.

Method raiseSalary() pada Manager dioverride untuk memberikan tambahan bonus berdasarkan lama bekerja.

```
@Override  
public void raiseSalary(double byPercent) {  
  
    java.util.GregorianCalendar todaysDate =  
        new java.util.GregorianCalendar();  
  
    int currentYear =  
        todaysDate.get(java.util.Calendar.YEAR);  
  
    double bonus =  
        0.5 * (currentYear - hireYear());
```

```

        super.raiseSalary(byPercent + bonus);
    }

```

Hasil Implementasi:

```

PS C:\Users\hsyyn\Downloads\Kuliah\Smt 3\PB0\W4 PB0\Exercise 2> cd "c:\Users\hsyyn\Downloa
ds\Kuliah\Smt 3\PB0\W4 PB0\Exercise 3\" ; if ($?) { javac ManagerTest.java } ; if ($?) { j
ava ManagerTest }
Sebelum sorting:
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993
Setelah sorting:
Antonio Rossi 2100000.0 1989
Maria Bianchi 3062500.0 1991
Isabel Vidal 3150000.0 1993

```

5. Percobaan 3(2) - Sortable dan Employee

Percobaan ini bertujuan untuk memahami penggunaan abstract class dan melakukan sorting terhadap object Employee.

5.1 Sortable

Class Sortable dibuat sebagai abstract class.

```

public abstract class Sortable {

    public abstract int compare(Sortable b);

    public static void shell_sort(Sortable[] a) {
        // proses sorting
    }
}

```

Method compare() bersifat abstract sehingga class turunannya harus menentukan bagaimana object dibandingkan.

5.2 Employee

Employee kemudian dibuat sebagai subclass dari Sortable.

```

public class Employee extends Sortable

```

Method compare() digunakan untuk membandingkan salary.

```

@Override
public int compare(Sortable b) {

    Employee eb = (Employee) b;

```

```

        if (salary < eb.salary) {
            return -1;
        }

        if (salary > eb.salary) {
            return 1;
        }

        return 0;
    }
}

```

Artinya:

- -1 -> salary employee pertama lebih kecil
- 1 -> salary employee pertama lebih besar
- 0 -> salary keduanya sama

Sorting Employee

Sorting dilakukan menggunakan:

```
Sortable.shell_sort(staff);
```

Object Employee diurutkan berdasarkan salary dari yang terkecil ke yang terbesar.

Hasil Implementasi:

```

PS C:\Users\hsyyn\Downloads\Kuliah\Smt 3\PBO\W4 PBO\Exercise 3> cd "c:\Users\hsyyn\Downloa
ds\Kuliah\Smt 3\PBO\W4 PBO\Exercise 3\" ; if ($?) { javac EmployeeTest.java } ; if ($?) {
java EmployeeTest }
Sebelum sorting:
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993

Setelah sorting:
Antonio Rossi 2100000.0 1989
Maria Bianchi 2625000.0 1991
Isabel Vidal 3150000.0 1993

```

6. Analisis

Dari percobaan yang dilakukan, inheritance membuat class turunan dapat menggunakan kembali atribut dan method dari superclass.

Pada Cylinder, inheritance digunakan untuk mendapatkan fitur dari Circle. Method `getArea()` dioverride karena perhitungan luas cylinder berbeda dengan circle.

Pada Shape, inheritance digunakan pada beberapa bentuk seperti Circle dan Rectangle. Kemudian Square mewarisi Rectangle, sehingga terbentuk inheritance bertingkat.

Pada Employee dan Manager, overriding digunakan agar Manager dapat memiliki implementasi `raiseSalary()` yang berbeda dari Employee.

Sedangkan pada Sortable, abstract method compare() digunakan untuk menentukan aturan perbandingan object. Pada Employee, aturan tersebut ditentukan berdasarkan nilai salary.

7. Hasil Praktikum

Percobaan	Hasil
Circle -> Cylinder	Berhasil menerapkan inheritance dan overriding
Shape -> Circle	Berhasil menerapkan inheritance
Shape -> Rectangle	Berhasil menerapkan inheritance
Rectangle -> Square	Berhasil menerapkan inheritance bertingkat
Employee -> Manager	Berhasil menerapkan overriding
Sortable -> Employee	Berhasil menerapkan abstract class
Sorting Employee	Berhasil mengurutkan berdasarkan salary

8. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, konsep inheritance pada Java dapat digunakan untuk membuat class yang memiliki hubungan dengan class lainnya.

Penggunaan extends memungkinkan subclass mewarisi atribut dan method dari superclass. Keyword super() digunakan untuk memanggil constructor superclass, sedangkan super.method() dapat digunakan untuk memanggil method milik superclass.

Selain itu, overriding memungkinkan subclass memiliki implementasi method yang berbeda sesuai kebutuhannya. Abstract class Sortable juga dapat digunakan sebagai dasar untuk membuat object yang dapat dibandingkan.

Pada percobaan Employee, method compare() digunakan untuk menentukan perbandingan berdasarkan salary sehingga data employee dapat diurutkan menggunakan shell_sort().

9. Lesson Learned

Dari praktikum ini ada beberapa hal yang dapat dipelajari:

- Inheritance mempermudah penggunaan kembali kode. Pada percobaan Cylinder dan Circle, subclass dapat memakai atribut dan method dari superclass tanpa perlu menuliskannya ulang. Subclass hanya perlu menambahkan bagian yang membedakannya.
- Keyword super berperan penting dalam hubungan antar class. super() digunakan untuk memanggil constructor superclass, sedangkan super.method() digunakan untuk memanggil method milik superclass. Pada Cylinder, super.getArea() dimanfaatkan untuk memperoleh luas alas tanpa menghitung ulang rumus luas lingkaran.
- Overriding digunakan ketika perilaku subclass perlu berbeda dari superclass. Hal ini terlihat pada getArea() di Cylinder, raiseSalary() di Manager, dan setWidth() di Square. Pada Square, overriding menjaga agar panjang dan lebar selalu bernilai sama.

- d. Abstract class berfungsi sebagai aturan dasar bagi subclass. Method abstract `compare()` pada `Sortable` mewajibkan subclass menentukan sendiri cara membandingkan object. Dengan demikian, proses sorting cukup dibuat satu kali dan dapat dipakai untuk berbagai jenis object dengan kriteria yang berbeda.